

Day 5: Object-Oriented Programming

AI and Machine Learning Course

Sandesh Dhital

Nesfield International College

July 25, 2026

Today's Agenda

- 1 Introduction to OOP
- 2 Six Pillars of OOP
- 3 Classes and Objects
- 4 Methods
- 5 Inheritance
- 6 Practical OOP Patterns
- 7 Practice

What is Object-Oriented Programming?

- A programming paradigm based on **objects**
- Objects bundle **data** (attributes) and **behavior** (methods) together
- Models real-world entities in code
- Python is a **multi-paradigm** language (supports OOP, functional, procedural)

Key Concepts

Class = Blueprint/Template — **Object** = Instance of a class

Why Use OOP?

- 1 **Organization** – Group related data and functions together
- 2 **Reusability** – Write once, create many objects
- 3 **Modularity** – Easy to maintain and debug
- 4 **Abstraction** – Hide complexity, expose simple interfaces
- 5 **Inheritance** – Build on existing code without rewriting

Real-World Analogy

Class: “Car” (blueprint)

Objects: Your car, my car, taxi (specific instances)

1. Class

- A **class** is a **blueprint** or **template** for creating objects
- Defines the **attributes** (data) and **methods** (behavior) that objects will have
- Does **not** occupy memory for data until an object is created

```
class Car:
    def __init__(self, brand, color):
        self.brand = brand
        self.color = color

    def drive(self):
        return f"{self.brand} is driving!"
```

Analogy

A class is like an **architectural blueprint** – it describes the structure, but is not a house itself.

2. Object

- An **object** is a specific **instance** of a class
- Each object has its own **unique data** but shares the class's methods
- You can create **multiple objects** from the same class

```
# Creating objects (instances) from the Car class
car1 = Car("Toyota", "Red")
car2 = Car("Honda", "Blue")

print(car1.brand)      # Toyota
print(car2.brand)      # Honda
print(car1.drive())    # Toyota is driving!
```

Key Point

Class = Blueprint → **Object** = Real thing built from that blueprint.
car1 and car2 are two different objects of the same class.

3. Inheritance

- A child class **inherits** attributes and methods from a parent class
- Promotes **code reuse** – no need to rewrite common functionality
- Child class can **add new** features or **override** parent behavior

Types

Single, Multiple, Multilevel, Hierarchical, Hybrid

```
class Animal:                                # Parent
    def __init__(self, name):
        self.name = name
    def speak(self):
        return "Some sound"

class Dog(Animal):                            # Child
    def speak(self):
        return f"{self.name}: Woof!"

dog = Dog("Buddy")
print(dog.speak())
# Buddy: Woof!
print(dog.name)
# Buddy (inherited)
```

4. Abstraction

- **Hiding complex details** and showing only the **essential features**
- Users interact with a **simple interface** without knowing internal workings
- Achieved using **abstract classes** (cannot be instantiated directly)

Analogy

A **TV remote**: you press buttons (interface) without knowing the circuit inside.

```
from abc import ABC, abstractmethod

class Shape(ABC):      # Abstract
    @abstractmethod
    def area(self):     # Must implement
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius
    def area(self):
        return 3.14 * self.radius ** 2

c = Circle(5)
print(c.area())      # 78.5
# s = Shape()        # ERROR!
```


5. Polymorphism

- “**Many forms**” – same method name, **different behavior** in different classes
- Allows writing code that works with objects of **different types**
- Achieved through **method overriding** and **duck typing** in Python

Duck Typing

“If it walks like a duck and quacks like a duck, it *is* a duck.”

```
class Cat:
    def speak(self):
        return "Meow!"

class Dog:
    def speak(self):
        return "Woof!"

class Duck:
    def speak(self):
        return "Quack!"

# Same name, different behavior
for animal in [Cat(), Dog(), Duck()]:
    print(animal.speak())
```

6. Encapsulation

- **Bundling data and methods** together inside a class
- **Restricting direct access** to internal data using private attributes
- Access controlled through **getter/setter** methods

Access Modifiers

name = Public

_name = Protected

__name = Private

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):
        return self.__balance

acc = BankAccount(1000)
acc.deposit(500)
print(acc.get_balance()) # 1500
# acc.__balance # ERROR!
```

Defining a Class

```
class Dog:
    # Class attribute
    species = "Canis familiaris"

    # Constructor
    def __init__(self, name, age):
        self.name = name
        self.age = age

    # Instance method
    def bark(self):
        return f"{self.name}: Woof!"
```

```
# Creating objects
dog1 = Dog("Buddy", 3)
dog2 = Dog("Max", 5)

print(dog1.bark())
# Buddy: Woof!

print(dog1.species)
# Canis familiaris
```

The `__init__` Method (Constructor)

- Called **automatically** when creating a new object
- Initializes the object's attributes
- `self` refers to the **current instance**

```
class Student:
    def __init__(self, name, age, grade):
        self.name = name      # Instance attribute
        self.age = age        # Instance attribute
        self.grade = grade    # Instance attribute

# Creating a student object
s1 = Student("Alice", 20, "A")
print(s1.name)      # Alice
print(s1.age)       # 20
```

Class vs Instance Attributes

```
class Car:
    # Class attribute (shared by ALL cars)
    wheels = 4

    def __init__(self, brand, color):
        # Instance attributes (unique to each car)
        self.brand = brand
        self.color = color

car1 = Car("Toyota", "Red")
car2 = Car("Honda", "Blue")

print(car1.wheels)    # 4 (class attribute)
print(car1.brand)     # Toyota (instance attribute)
print(car2.brand)     # Honda (instance attribute)
```

Rule

Class attributes are shared; instance attributes are unique to each object.

Instance Methods

```
class Circle:
    pi = 3.14159

    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return Circle.pi * self.radius **
            2

    def circumference(self):
        return 2 * Circle.pi * self.radius

    def resize(self, new_radius):
        self.radius = new_radius
```

```
c = Circle(5)
print(c.area())
# 78.53975

print(c.circumference())
# 31.4159

c.resize(10)
print(c.area())
# 314.159
```

Special Methods: `__str__` and `__repr__`

```
class Book:
    def __init__(self, title, author, pages):
        self.title = title
        self.author = author
        self.pages = pages

    def __str__(self):
        return f"'{self.title}' by {self.author}"

    def __repr__(self):
        return f"Book('{self.title}', '{self.author}', {self.pages})"

book = Book("Python Crash Course", "Eric Matthes", 544)
print(book)           # 'Python Crash Course' by Eric Matthes
print(repr(book))     # Book('Python Crash Course', 'Eric Matthes', 544)
```

Understanding Inheritance

- A class can **inherit** attributes and methods from another class
- **Parent class** (base/super class) – the class being inherited from
- **Child class** (derived/sub class) – the class that inherits
- Promotes **code reuse** and establishes relationships

Inheritance Hierarchy Example

Animal → Dog, Cat

Shape → Circle, Rectangle

BankAccount → SavingsAccount, CheckingAccount

Inheritance in Python

```
class Animal:
    def __init__(self, name, sound):
        self.name = name
        self.sound = sound

    def speak(self):
        return f"{self.name}: {self.
            sound}!"

class Dog(Animal):
    def fetch(self):
        return f"{self.name} fetches!"
```

```
class Cat(Animal):
    def purr(self):
        return f"{self.name} purrs
            ..."

dog = Dog("Buddy", "Woof")
print(dog.speak())
# Buddy: Woof!
print(dog.fetch())
# Buddy fetches!
```

Method Overriding & super()

```
class Shape:
    def __init__(self, color):
        self.color = color
    def area(self):
        return 0    # Base

class Rectangle(Shape):
    def __init__(self, color, w, h):
        super().__init__(color)
        self.width = w
        self.height = h
    def area(self):    # Override
        return self.width * self.height
```

```
rect = Rectangle("blue", 5, 3)

print(rect.color)
# blue (inherited)

print(rect.area())
# 15 (overridden)
```

Encapsulation (Data Hiding)

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.__balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
            return amount
        return 0
```

```
def get_balance(self):
    return self.__balance

acc = BankAccount("Alice", 1000)
acc.deposit(500)
print(acc.get_balance())
# 1500
```

Composition (Has-A Relationship)

```
class Engine:
    def __init__(self, horsepower):
        self.horsepower = horsepower

    def start(self):
        return "Engine started!"

class Car:
    def __init__(self, brand, horsepower):
        self.brand = brand
        self.engine = Engine(horsepower)    # Car HAS an Engine

    def start(self):
        return f"{self.brand}: {self.engine.start()}"

car = Car("Toyota", 150)
print(car.start())    # Toyota: Engine started!
print(car.engine.horsepower)    # 150
```

Practice: Bank Account Class

```
class BankAccount:
    def __init__(self, owner, balance
=0):
        self.owner = owner
        self.__balance = balance
        self.transactions = []

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            self.transactions.append(
                f"Deposit: +{amount}")
```

```
    def withdraw(self, amount):
        if 0 < amount <= self.
            __balance:
                self.__balance -= amount
                self.transactions.append
                    (
                        f"Withdraw: -{amount
                            }")
                return amount
        print("Insufficient funds!")
        return 0

    def get_balance(self):
        return self.__balance
```

Practice: Savings Account (Inheritance)

```
class SavingsAccount(BankAccount):
    def __init__(self, owner, balance=0, interest_rate=0.05):
        super().__init__(owner, balance)
        self.interest_rate = interest_rate

    def apply_interest(self):
        interest = self.get_balance() * self.interest_rate
        self.deposit(interest)
        return interest

savings = SavingsAccount("Bob", 10000, 0.08)
interest = savings.apply_interest()
print(f"Interest earned: Rs.{interest}")
print(f"New balance: Rs.{savings.get_balance()}")
```

Summary

- ❶ **Classes & Objects:** Blueprint vs instances
- ❷ **__init__:** Constructor to initialize attributes
- ❸ **Methods:** Functions that belong to a class
- ❹ **Inheritance:** Child classes inherit from parent classes
- ❺ **Encapsulation:** Hide internal data with private attributes
- ❻ **Composition:** Objects containing other objects

Homework

Create a Library system with Book and Member classes. Members can borrow and return books. Track which books are available and which are borrowed.